Experiment 4 – Exploration vs Exploitation

NAME:N.AKSHAYA

REG.NO:192425129

COURSE CODE:MLA0305

COURSE NAME:REINFORCEMENT LEARNING

SOLT:B

4.Exploration vs Exploitation Analysis using Multi-Armed Bandit Algorithms

Aim:

To implement the **Multi-Armed Bandit (MAB)** algorithm using the **Epsilon-Greedy strategy** and analyze the trade-off between **Exploration** and **Exploitation** through summary statistics and graphical visualization.

Procedure:

1. Define the reward probabilities for multiple slot machine arms.
2. Initialize reward estimates and action counts.
3. Choose an exploration rate (ε).
4. At each iteration:
    - Explore by selecting a random arm with probability ε.
    - Exploit by selecting the arm with the highest estimated reward with probability (1−ε).
5. Receive a reward from the selected arm.
6. Update the estimated reward using the sample average method.
7. Repeat the process for a fixed number of iterations.

8. Display the summary of exploration, exploitation, rewards, and estimated values.
9. Visualize the results using graphs.

Algorithm:

Epsilon-Greedy Selection

- With probability **ε**, choose a random action (Exploration).
- With probability **1−ε**, choose the best-known action (Exploitation).

Python code:

```
import numpy as np

import matplotlib.pyplot as plt

# Multi-Armed Bandit Parameters

np.random.seed(42)

arms = ["Arm 1", "Arm 2", "Arm 3", "Arm 4", "Arm 5"]

true_rewards = [0.2, 0.5, 0.7, 0.4, 0.9]

epsilon = 0.2

episodes = 1000

# Initialization

Q = np.zeros(len(arms))

N = np.zeros(len(arms))

total_reward = 0

reward_history = []

exploration = 0

exploitation = 0

# Epsilon-Greedy Algorithm

for i in range(episodes):
```

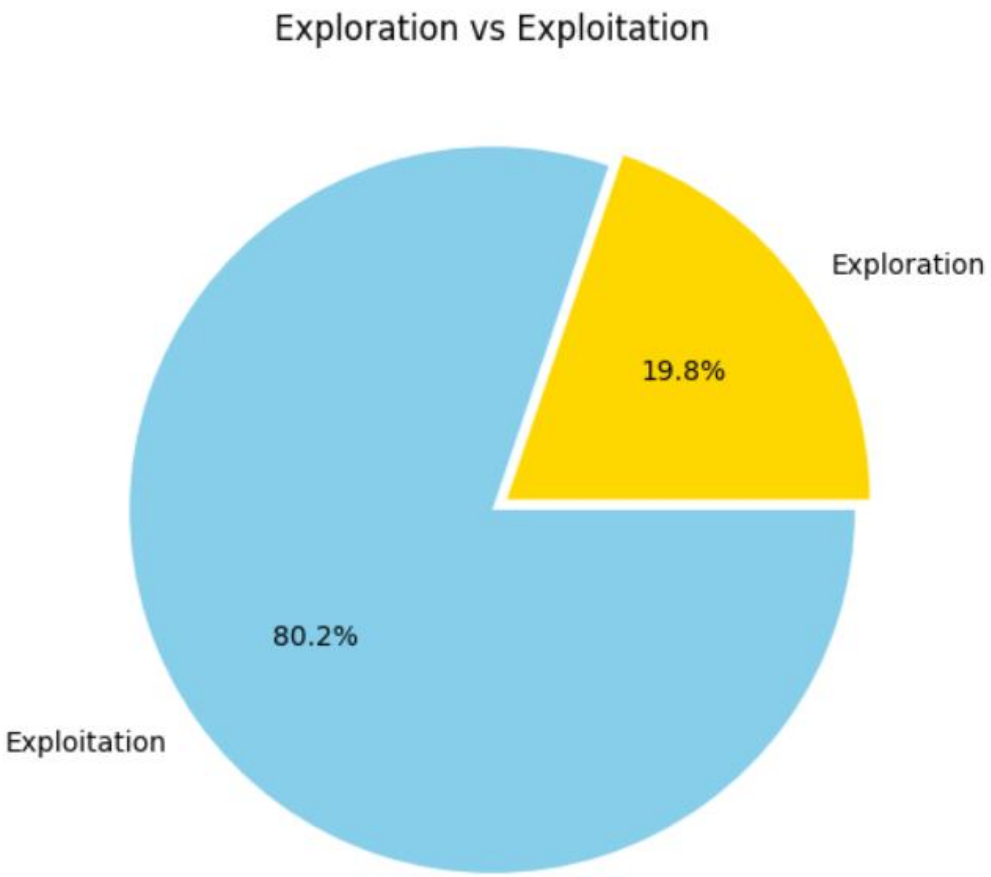
```
    if np.random.rand() < epsilon:
        action = np.random.randint(len(arms))
        exploration += 1
    else:
        action = np.argmax(Q)
        exploitation += 1
    reward = 1 if np.random.rand() < true_rewards[action] else 0
    total_reward += reward
    N[action] += 1
    Q[action] = Q[action] + (reward - Q[action]) / N[action]
    reward_history.append(total_reward)
# Summary
print("="*50)
print("MULTI-ARMED BANDIT SUMMARY")
print("="*50)
print(f"\nTotal Episodes      : {episodes}")
print(f"Exploration Count   : {exploration}")
print(f"Exploitation Count  : {exploitation}")
print(f"Total Reward        : {total_reward}")
print(f"Average Reward      : {total_reward/episodes:.3f}")
print("\nEstimated Value of Each Arm\n")
for i in range(len(arms)):
    print(f"{arms[i]} : {Q[i]:.3f}")
```

```
best_arm = arms[np.argmax(Q)]

print("\nBest Arm Selected :", best_arm)

# Visualization

plt.figure(figsize=(6,6))

plt.pie([exploration, exploitation],

    labels=["Exploration","Exploitation"],

    autopct='%1.1f%%',

    colors=['gold','skyblue'],

    explode=(0.05,0))

plt.title("Exploration vs Exploitation")

plt.show()
```

Result:

The **Multi-Armed Bandit** algorithm using the **Epsilon-Greedy strategy** was successfully implemented. The agent balanced **exploration** by trying different arms and **exploitation** by selecting the arm with the highest estimated reward. Over multiple episodes, the algorithm identified the optimal arm with the highest expected reward. The summary statistics and visualizations clearly demonstrate the exploration–exploitation trade-off and the learning behavior of the agent.

Summary Result:

```
==================================================
MULTI-ARMED BANDIT SUMMARY
==================================================

Total Episodes      : 1000
Exploration Count   : 198
Exploitation Count  : 802
Total Reward        : 819
Average Reward      : 0.819
```

Visualization Result: